

# Experiment - 8

Student Name: Bharti Mehra  
 Branch: CSE - AIML  
 Semester: 5  
 Subject Name: ADBMS

UID: 24BAI70051  
 Section/Group: 24AIT\_KRG-1\_B  
 Date of Performance: 7/9/26  
 Subject Code: 24CSP-310

## Q1.

A company maintains customer information, product information, and sales order information in three separate tables: customer\_master, product\_catalog, and sales\_orders.

The company wants to provide an order report to an external client. The report must contain the Order ID, Order Date, Customer Name, Product Name, and Final Amount.

For every order, first calculate the Gross Amount using:

Gross Amount = Unit Price × Quantity

Then calculate the Discount Amount using:

Discount Amount = Gross Amount × Discount Percentage / 100

Finally, calculate the Final Amount using:

Final Amount = Gross Amount – Discount Amount

Create a complex view using the three tables to generate this report. The view should expose only the required five columns and should not expose the customer's phone/email, product brand/unit price, or the order's quantity/discount percentage. The company also wants to provide access to an external client. Create a PostgreSQL user named client\_user and give this user only SELECT permission on the view. The client must not have direct access to the three original tables and must not be allowed to modify the view data.

Output:

| order_id | order_date | full_name   | product_name     | final_amount |
|----------|------------|-------------|------------------|--------------|
| 1        | 2025-09-01 | Amit Sharma | Smartphone X100  | 47500.00     |
| 2        | 2025-09-02 | Priya Verma | Laptop Pro 15    | 58500.00     |
| 3        | 2025-09-03 | Ravi Kumar  | Wireless Earbuds | 15000.00     |
| 4        | 2025-09-04 | Neha Singh  | Smartwatch Fit   | 27600.00     |
| 5        | 2025-09-05 | Arjun Mehta | Gaming Console   | 39600.00     |
| 6        | 2025-09-06 | Priya Verma | Smartphone X100  | 23750.00     |
| 7        | 2025-09-07 | Amit Sharma | Wireless Earbuds | 10000.00     |

**SOLUTION:**

```
CREATE OR REPLACE VIEW vW_FINAL_AMOUNT_CALCULATION  
AS
```

```
SELECT
```

```
    O.order_id,  
    O.order_date,  
    C.full_name,  
    P.product_name,  
    (P.unit_price * O.quantity)  
-  
    (  
        (P.unit_price * O.quantity)  
        * O.discount_percent / 100  
    ) AS final_amount
```

```
FROM customer_master C
```

```
JOIN sales_orders O
```

```
    ON C.customer_id = O.customer_id
```

```
JOIN product_catalog P
```

```
    ON P.product_id = O.product_id;
```

```
SELECT * FROM vW_FINAL_AMOUNT_CALCULATION;
```

```
CREATE USER CLIENT_123
```

```
PASSWORD '12345';
```

```
GRANT SELECT ON vW_FINAL_AMOUNT_CALCULATION TO  
CLIENT_123;
```

**Q2.**

An e-commerce company wants to regularly analyze the performance of its product categories. The company stores product, category, customer, and order information in separate tables. To generate the category performance report, the database needs to perform multiple JOINS, GROUP BY, aggregate functions, DISTINCT counting, calculated values, and CASE statements. Since the same report is generated frequently, executing the complete analytical query every time can increase database workload.

Create a PostgreSQL Materialized View named: mv\_category\_sales

Also demonstrate that when new order data is inserted, the Materialized View does not automatically reflect the change and must be refreshed.

Finally, use EXPLAIN ANALYZE to compare the performance of the original analytical query with the Materialized View query.

Output:

| category_id | category_name | total_products_sold | total_orders | unique_customers | total_revenue | avg_revenue_per_order | category_performance |
|-------------|---------------|---------------------|--------------|------------------|---------------|-----------------------|----------------------|
| C1          | Electronics   | 5                   | 4            | 3                | 12000         | 3000.00               | Excellent            |
| C2          | Clothing      | 4                   | 3            | 2                | 10000         | 3333.33               | Excellent            |
| C3          | Books         | 3                   | 1            | 1                | 1500          | 1500.00               | Low                  |

### SOLUTION:

EXPLAIN ANALYZE

```

SELECT CAT.CATEGORY_ID, CAT.CATEGORY_NAME,
       SUM(O.QUANTITY) AS TOTAL_PRODUCTS_SOLD,
       COUNT(DISTINCT O.ORDER_ID) AS TOTAL_ORDERS,
       COUNT(DISTINCT O.CUSTOMER_ID) AS UNIQUE_CUSTOMERS,
       SUM(O.QUANTITY*O.UNIT_PRICE) AS TOTAL_REVENUE,
       ROUND(SUM(O.QUANTITY*O.UNIT_PRICE)/COUNT(DISTINCT
O.ORDER_ID), 2) AS AVERAGE_REVENUE_PER_ORDER,
       CASE
           WHEN SUM(O.QUANTITY*O.UNIT_PRICE) >= 10000 THEN
'EXCELLENT'
           WHEN SUM(O.QUANTITY*O.UNIT_PRICE) >= 6000 THEN
'GOOD'
           WHEN SUM(O.QUANTITY*O.UNIT_PRICE) >=3000 THEN
'AVERAGE'
           ELSE 'LOW'
       END AS CATEGORY_PERFORMANCE
FROM ORDER_DETAILS AS O
JOIN PRODUCT_MASTER AS P
ON O.PRODUCT_ID=P.PRODUCT_ID
JOIN CATEGORY_MASTER AS CAT
ON P.CATEGORY_ID=CAT.CATEGORY_ID
GROUP BY CAT.CATEGORY_ID, CAT.CATEGORY_NAME;

```

```

CREATE MATERIALIZED VIEW MV_CATEGORY_SALE
AS

```

```

    SELECT
    c.category_id,

```

```
c.category_name,  
SUM(od.quantity) AS total_products_sold,  
COUNT(DISTINCT od.order_id) AS total_orders,  
COUNT(DISTINCT od.customer_id) AS unique_customers,  
SUM(od.quantity * od.unit_price) AS total_revenue,  
ROUND(  
    SUM(od.quantity * od.unit_price)  
    / COUNT(DISTINCT od.order_id),  
    2  
) AS avg_revenue_per_order,  
CASE  
    WHEN SUM(od.quantity * od.unit_price) >= 10000  
        THEN 'Excellent'  
    WHEN SUM(od.quantity * od.unit_price) >= 6000  
        THEN 'Good'  
    WHEN SUM(od.quantity * od.unit_price) >= 3000  
        THEN 'Average'  
    ELSE 'Low'  
END AS category_performance  
FROM category_master c  
JOIN product_master p  
    ON c.category_id = p.category_id  
JOIN order_details od  
    ON p.product_id = od.product_id  
GROUP BY  
    c.category_id,  
    c.category_name;  
  
EXPLAIN ANALYZE  
SELECT * FROM MV_CATEGORY_SALE;  
REFRESH MATERIALIZED VIEW MV_CATEGORY_SALE;
```